

C.A.S.E Pharmacy

Aracely Gomez - ARACELY.GOMEZ@BARUCHMAIL.CUNY.EDU

Allison Malin – ALLISON.MALIN@BARUCHMAIL.CUNY.EDU

Chelsea Tanchez - CHELSEA.TANCHEZ@BARUCHMAIL.CUNY.EDU

Emma Parvar - EMMA.PARVAR@BARUCHMAIL.CUNY.EDU

Shahin Akter - SHAHIN.AKTER1@BARUCHMAIL.CUNY.EDU

CIS 3400 Section 25611

Group 4

Roles:

Aracely Gomez: *Information Analyst*

Allison Malin: *Information Provider/Data Gatherer*

Chelsea Tanchez: *Document Writer*

Emma Parvar: *System Analyst*

Shahin Akter: *Data Analyst*

I. Business Scenario

Our company has owned and operated a local pharmacy in midtown Manhattan for 5 years. We have been using spreadsheets and verbal communication to manage patients' prescription pickups and balance customer flows daily. We would like to replace this manual method of tracking the business with a database.

In our business, patients receive their prescriptions from our pharmacy. Customers make appointments with their pharmacist or other staff and can receive a number of services such as prescription dispensing, medication consultations, refill requests, and insurance processing. We need to keep track of medications including the drug name, dosage, quantity, lot number, expiration date, and national drug code. While each medication has a standard price, insurance coverage, promotions, and other factors may affect the actual price charged to the customer for a given prescription.

We also need to keep track of our Employees including their home address, contact information, and rate of pay. We need to keep track of each Customer's contact information as well as their date of birth, allergies, and social security number. For prescription pickups, we need to know the date and time of the pickup, which prescription is being picked up, and who is picking it up.

Identification:

Since the issue our business is having is due to the pick-up system, some ways to help improve this area are through collecting certain information. We can start by collecting the patient's information such as full name, date of birth, phone number, or address to know if we have the patient in our system or if this is the first time they are picking up medication in our pharmacy. In addition, Staff can check the date filled field to confirm if a prescription is ready for pickup and use refills remaining to see if a refill is allowed. When a patient arrives, we can quickly verify which Prescription they are picking up and record the exact filled date as their pickup time. This creates a digital log of when every prescription is claimed, solving the issue of unclaimed medications and providing proof of pickup.

The system also should require the prescriber's information such as name, the clinic they are associated with, and a contact number in that way it allows the pharmacy to verify prescription and get in contact with them if there are any questions. When collecting the prescriber's information, we should also collect the medication information such as the medication name, quantity prescribed, directions for use, refills allowed and the dosage form in order to make sure the prescriptions are filled correctly, quick to locate, proper dispensing and know if they're in stock.

The system must also include prescription status information such as whether the prescription has been received, processed ready for pick up or has already been picked up and when it has been picked up it should collect the date, time and name of the person who picked up the prescription that way if there are any issue related to unclaimed or delayed prescription there is proof that can be provided.

For smooth operations and reducing waiting times, we will use the Patient and Insurance Plan data. By storing Patient details, we can instantly retrieve their profile and link Insurance Plan information (such as plan type, member ID and co-payment) at the counter. This eliminates the need for customers to provide the same information repeatedly, speeding up transactions.

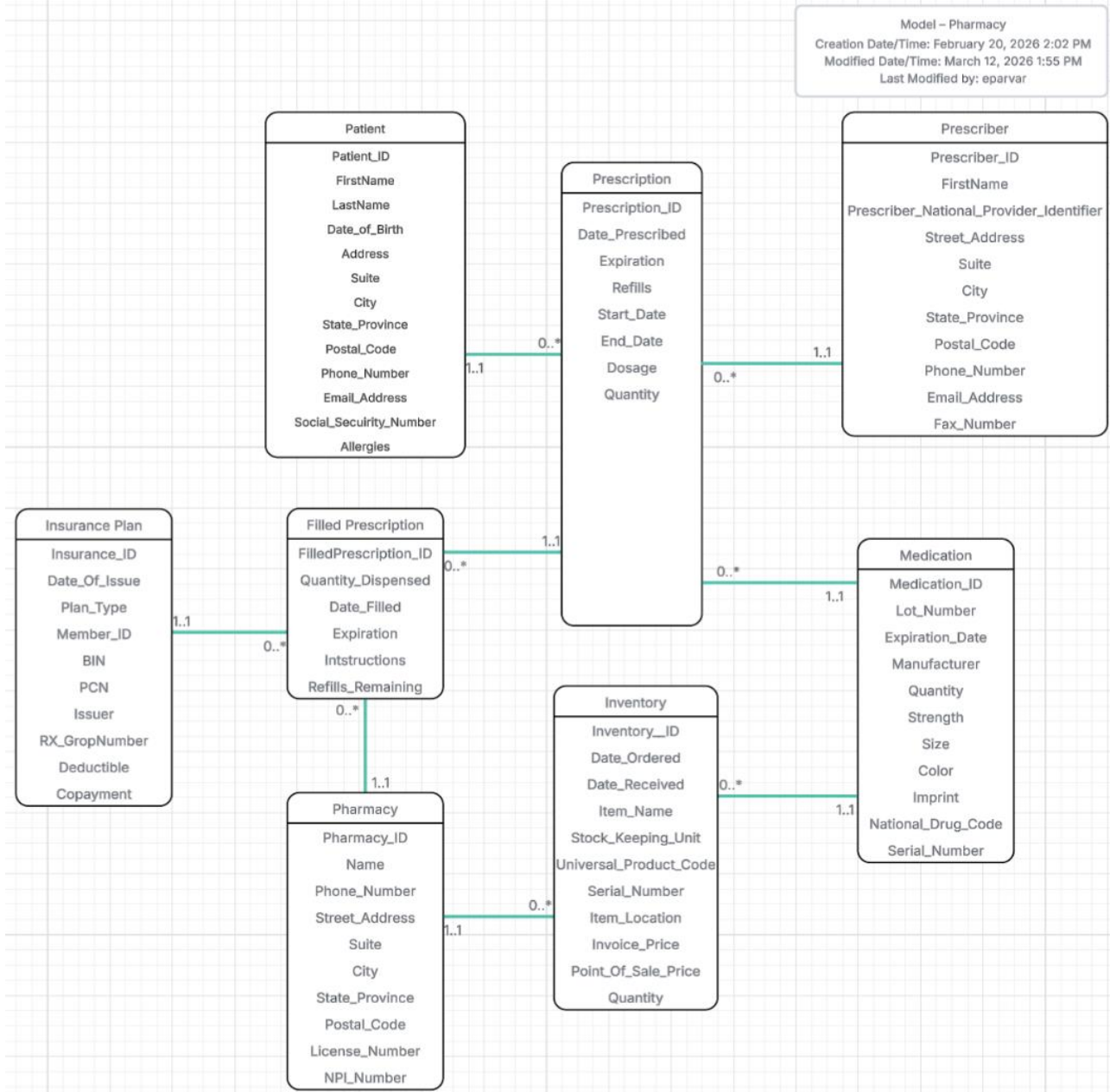
With real-time inventory tracking, staff can check if the medication is in stock as soon as the prescription is written. This helps avoid delays with items that need to be ordered and makes pickup faster and easier for patients.

Commentary

Based on the above description, construct an Entity Relationship model using UML notation that will capture all the business data needs.

II. ER Model using UML Notation

Based on the above description, we develop the following Entity Relationship model using UML notation.



Commentary

What we like about this model:

- All relationship lines run horizontally or vertically with no lines crossing, keeping the diagram clean and easy to follow.
- Attribute names are fully spelled out with no abbreviations or spaces, such as FilledPrescription_ID and Prescriber_National_Provider_Identifier, leaving no ambiguity.
- Each relationship has two verb phrases so we can read it in both directions and form clear relationship sentences.
- The diagram includes a legend in the upper-right corner with the model's name, creation date, and last modifier, making version tracking straightforward.

Relationship Sentences

One prescription must belong to one patient.
One patient may have many prescriptions.

One prescription must be authorized by one prescriber.
One prescriber may authorize many prescriptions.

One prescription must specify one medication.
One medication may be included in many prescriptions.

One prescription may result in many filled prescriptions.
One filled prescription must be associated with one prescription.

One filled prescription must be claimed by one insurance plan.
One insurance plan may cover many filled prescriptions.

One filled prescription must be dispensed by one pharmacy.
One pharmacy may dispense many filled prescriptions.

One pharmacy may maintain many inventories.
One inventory must be maintained by one pharmacy.

One inventory must contain one medication.
One medication may be recorded in many inventories.

Commentary

What we like about these relationship sentences:

- Every relationship is expressed in both directions; The sentences correctly distinguish between “must” and “may” to reflect mandatory versus optional participation. For

example, a filled prescription must be associated with exactly one prescription, while a patient may have many prescriptions but is not required to have any on record.

- Each sentence uses a precise and meaningful verb phrase such as “authorized by,” “dispensed by,” and “maintained by” that clearly describes the nature of the relationship rather than using generic language like “related to” or “connected with.”

The sentences are organized by relationship pairs, making it straightforward to trace each one back to the corresponding connection in the ER diagram. With the ER model finalized, we move on to the next step – to convert the conceptual ER model to a logical Relational model.

III. Conversion to Relational Model

Patient (Patient_ID (Key), FirstName, LastName, Date_of_Birth, Address, Suite, City, State_Province, Postal_Code, Phone_Number, Email_Address, Social_Security_Number, Allergies)

Prescriber (Prescriber_ID (Key), FirstName, Prescriber_National_Provider_Identifier, Street_Address, Suite, City, State_Province, Postal_Code, Phone_Number, Email_Address, Fax_Number)

Medication (Medication_ID (Key), Lot_Number, Expiration_Date, Manufacturer, Quantity, Strength, Size, Color, Imprint, National_Drug_Code, Serial_Number)

Prescription (Prescription_ID (Key), Patient_ID (FK), Prescriber_ID (FK), Medication_ID (FK), Date_Prescribed, Expiration, Refills, Start_Date, End_Date, Dosage, Quantity)

Insurance Plan (Insurance_ID (Key), Date_Of_Issue, Plan_Type, Member_ID, BIN, PCN, Issuer, RX_GroupNumber, Deductible, Copayment)

Pharmacy (Pharmacy_ID (Key), Name, Phone_Number, Street_Address, Suite, City, State_Province, Postal_Code, License_Number, NPI_Number)

Filled Prescription (FilledPrescription_ID (Key), Prescription_ID (FK), Insurance_ID (FK), Pharmacy_ID (FK), Quantity_Dispensed, Date_Filled, Expiration, Instructions, Refills_Remaining)

Inventory (Inventory_ID (Key), Pharmacy_ID (FK), Medication_ID (FK), Date_Ordered, Date_Received, Item_Name, Stock_keeping_Unit, Universal_Product_Code, Serial_Number, Item_Location, Invoice_Price, Point_Of_Sale_Price, Quantity)

Commentary

- Keys are shown with the (key) designation and foreign keys are shown with the (fk) designation.

IV. Normalization

The steps to follow for each relation are:

1. Write out the relation including all attribute names. Indicate keys and foreign keys.
2. Provide some sample data for the relation.
3. State the key for the relation and write down all functional dependencies.
4. Go through the definitions of each normal form, starting with 1NF and working up to BCNF (or 3NF, depending on your project requirements).
5. If a relation meets the definition of a normal form, move to the next higher normal form.
6. If a relation fails to meet the definition of a normal form — for example, if it contains a partial-key dependency or a transitive dependency — split it into two new relations. Then begin the normalization process from the beginning with each of those two new relations.

Patient Relation

Patient (Patient_ID (Key), FirstName, LastName, Date_of_Birth, Address, Suite, City, State_Province, Postal_Code, Phone_Number, Email_Address, Social_Security_Number, Allergies)

Sample Data:

Patient_ID	FirstName	LastName	Date_of_Birth	Address	Suite	City	State_Province	Postal_Code	Phone_Number	Email_Address	Social_Security_Number	Allergies
P001	John	Smith	01/15/1985	123 Main St	Apt 2A	New York	NY	10001	555-123-4567	john.smith@email.com	123-45-6789	Penicillin
P002	Sarah	Johnson	03/22/1990	456 Oak Ave	Apt 3B	Garfield	NJ	7026	555-234-5678	sarah.j@email.com	234-56-7890	None
P003	Michael	Williams	07/08/1978	789 Elm St	Suite 1	Beverly Hills	CA	90210	555-345-6789	m.williams@email.com	345-67-8901	Aspirin
P004	Emily	Brown	11/30/1995	321 Pine Rd	Apt 5C	Chicago	IL	60601	555-456-7890	emily.b@email.com	456-78-9012	Nuts
P005	David	Garcia	09/14/1965	654 Maple Dr	Unit 7	Houston	TX	77001	555-567-8901	d.garcia@email.com	567-89-0123	None

Key: Patient_ID

FD1: Patient_ID → FirstName, LastName, Date_of_Birth, Address, Suite, City, State_Province, Postal_Code, Phone_Number, Email_Address, Social_Security_Number, Allergies

FD2: Postal_Code → City, State_Province

1NF: Meets the definition of a relation

2NF: No partial key dependencies

3NF: Transitive dependency exists: Patient_ID → Postal_Code and Postal_Code → City, State_Province

Solution: Split Patient relation into two new relations named PatientData and Postal_Codes:

PatientData (Patient_ID (Key), FirstName, LastName, Date_of_Birth, Address, Suite, Postal_Code (fk), Phone_Number, Email_Address, Social_Security_Number, Allergies)

Key: Patient_ID

FD1: Patient_ID → FirstName, LastName, Date_of_Birth, Address, Suite, Postal_Code (fk), Phone_Number, Email_Address, Social_Security_Number, Allergies

1NF: Meets the definition of a relation
 2NF: No partial Key dependencies
 3NF: No transitive dependencies
 BCNF: All determinants are candidate keys

Postal_Codes (Postal_Code (key), City, State_Province)

Key: Postal_Code

FD1: Postal_Code → City, State_Province

1NF: Meets the definition of a relation

2NF: No partial key dependencies

3NF: No transitive dependencies

BCNF: All determinants are candidate keys

Prescriber Relation

Prescriber (Prescriber_ID (Key), FirstName, Prescriber_National_Provider_Identifier, Street_Address, Suite, City, State_Province, Postal_Code, Phone_Number, Email_Address, Fax_Number)

Sample Data:

Prescriber_ID	FirstName	LastName	Prescriber_National_Provider_Identifier	Street_Address	Suite	City	State_Province	Postal_Code	Phone_Number	Email_Address	Fax_Number
RX001	John	Smith	1234567890	100 Main St	Suite 1	Boston	MA	2115	555-100-1000	john.s@clinic.com	555-100-1001
RX002	Mary	Jones	2345678901	200 Oak Ave	Suite 2	Miami	FL	33101	555-200-2000	mary.j@clinic.com	555-200-2001
RX003	Bob	Brown	3456789012	300 Elm St	Suite 3	Austin	TX	78701	555-300-3000	bob.b@clinic.com	555-300-3001
RX004	Lisa	Davis	4567890123	400 Pine Rd	Suite 4	Denver	CO	80201	555-400-4000	lisa.d@clinic.com	555-400-4001
RX005	Tom	Wilson	5678901234	500 Maple Dr	Suite 5	Portland	OR	97201	555-500-5000	tom.w@clinic.com	555-500-5001

Key: Prescriber_ID

FD1: Prescriber_ID → FirstName, Prescriber_National_Provider_Identifier, Street_Address, Suite, City, State_Province, Postal_Code, Phone_Number, Email_Address, Fax_Number

FD2: Postal_Code → City, State_Province

1NF: Meets the definition of a relation

2NF: No partial key dependencies

3NF: Transitive dependency exists: Prescriber_ID → Postal_Code and Postal_Code → City, State_Province

Solution: Split Prescriber relation into two new relations named PrescriberData and Postal_Codes:

PrescriberData(Prescriber_ID (Key), FirstName, Prescriber_National_Provider_Identifier, Street_Address, Suite, Postal_Code (fk), Phone_Number, Email_Address, Fax_Number)

Key: Prescriber_ID

FD1: Prescriber_ID → FirstName, Prescriber_National_Provider_Identifier, Street_Address, Suite, Postal_Code (fk), Phone_Number, Email_Address, Fax_Number

1NF: Meets the definition of a relation

2NF: No partial key dependencies

3NF: No transitive dependencies

BCNF: All determinants are candidate keys

Medication Relation

Medication (Medication_ID (Key), Lot_Number, Expiration_Date, Manufacturer, Quantity, Strength, Size, Color, Imprint, National_Drug_Code, Serial_Number)

Sample Data:

Medication_ID	Lot_Number	Expiration_Date	Manufacturer	Quantity	Strength	Size	Color	Imprint	National_Drug_Code	Serial_Number
MED001	LOT12345	12/31/2026	PharmaCo	100	500mg	10mm	White	P500	12345-678-90	SN001234
MED002	LOT23456	06/30/2026	MediLab	50	250mg	8mm	Blue	M250	23456-789-01	SN002345
MED003	LOT34567	09/15/2026	HealthRx	75	100mg	12mm	Yellow	H100	34567-890-12	SN003456
MED004	LOT45678	03/20/2026	CurePharm	200	750mg	15mm	Pink	C750	45678-901-23	SN004567
MED005	LOT56789	11/10/2026	WellMed	150	50mg	6mm	Orange	W50	56789-012-34	SN005678

Key: Medication_ID

FD1: Medication_ID → Lot_Number, Expiration_Date, Manufacturer, Quantity, Strength, Size, Color, Imprint, National_Drug_Code, Serial_Number

1NF: Meets the definition of a relation

2NF: No partial Key dependencies

3NF: No transitive dependencies

Prescription Relation

Prescription (Prescription_ID (Key), Patient_ID (FK), Prescriber_ID (FK), Medication_ID (FK), Date_Prescribed, Expiration, Refills, Start_Date, End_Date, Dosage, Quantity)

Sample Data:

Prescription_ID	Patient_ID	Prescriber_ID	Medication_ID	Date_Prescribed	Expiration	Refills	Start_Date	End_Date	Dosage	Quantity
601000	P001	RX001	MED001	01/15/2025	01/15/2026	3	01/15/2025	02/15/2025	1 tablet daily	30
601001	P002	RX002	MED002	02/10/2025	02/10/2026	2	02/10/2025	03/10/2025	2 tablets daily	60
601002	P003	RX003	MED003	03/05/2025	03/05/2026	1	03/05/2025	04/05/2025	1 tablet twice daily	60
601003	P004	RX004	MED004	01/20/2025	01/20/2026	4	01/20/2025	02/20/2025	1 tablet daily	30
601004	P005	RX005	MED005	02/25/2025	02/25/2026	2	02/25/2025	03/25/2025	1 tablet every 12 hours	60

Key: Prescription_ID

FD1: Prescription_ID → Patient_ID (FK), Prescriber_ID (FK), Medication_ID (FK), Date_Prescribed, Expiration, Refills, Start_Date, End_Date, Dosage, Quantity

1NF: Meets the definition of a relation

2NF: No partial Key dependencies

3NF: No Transitive dependencies

BCNF: All determinants are candidate keys

Insurance Plan Relation

Insurance Plan (Insurance_ID (Key), Date_Of_Issue, Plan_Type, Member_ID, BIN, PCN, Issuer, RX_GroupNumber, Deductible, Copayment)

Sample Data:

Insurance_ID	Date_Of_Issue	Plan_Type	Member_ID	BIN	PCN	Issuer	RX_GroupNumber	Deductible	Copayment
INS001	01/01/2025	PPO	MEM123456		610014	Blue Cross	RX12345	1000	20
INS002	01/01/2025	HMO	MEM234567		610020	Aetna	RX23456	500	15
INS003	01/01/2025	EPO	MEM345678		610030	UnitedHealth	RX34567	1500	25
INS004	01/01/2025	PPO	MEM456789		610040	Cigna	RX45678	750	10
INS005	01/01/2025	HMO	MEM567890		610050	Humana	RX56789	300	20

Key: Insurance_ID

FD1: Insurance_ID → Date_Of_Issue, Plan_Type, Member_ID, BIN, PCN, Issuer, RX_GroupNumber, Deductible, Copayment

1NF: Meets the definition of a relation

2NF: No partial Key dependencies

3NF: No Transitive dependencies

BCNF: All determinants are candidate keys

Pharmacy Relation

Pharmacy (Pharmacy_ID (Key), Name, Phone_Number, Street_Address, Suite, City, State_Province, Postal_Code, License_Number, NPI_Number)

Sample Data:

Pharmacy_ID	Name	Phone_Number	Street_Address	Suite	City	State_Province	Postal_Code	License_Number	NPI_Number
PH001	Main Street Pharm	555-111-1111	100 Main St	Suite A	Boston	MA	2115	UC1001	9876543210
PH002	Oak Avenue Drugs	555-222-2222	200 Oak Ave	Suite B	Miami	FL	33101	UC2002	8765432109
PH003	Elm Street Pharma	555-333-3333	300 Elm St	Suite C	Austin	TX	78701	UC3003	7654321098
PH004	Pine Road Drugs	555-444-4444	400 Pine Rd	Suite D	Denver	CO	80201	UC4004	6543210987
PH005	Maple Drive Pharm	555-555-5555	500 Maple Dr	Suite E	Portland	OR	97201	UC5005	5432109876

Key: Pharmacy_ID

FD1: Pharmacy_ID → Name, Phone_Number, Street_Address, Suite, City, State_Province, Postal_Code, License_Number, NPI_Number

FD2 PostalCode → City, State_Province

1NF: Meets the definition of a relation

2NF: No partial key dependencies

3NF: Transitive dependency exists: Pharmacy_ID → Postal_Code and Postal_Code → City, State_Province

Solution: Split Pharmacy relation into two new relations named PharmacyData and Postal_Codes:

PharmacyData (Pharmacy_ID (Key), Name, Phone_Number, Street_Address, Suite, Postal_Code (FK), License_Number, NPI_Number)

Key: Pharmacy_ID

FD1: Pharmacy_ID → Name, Phone_Number, Street_Address, Suite, Postal_Code (FK), License_Number, NPI_Number

1NF: Meets the definition of a relation

2NF: No partial Key dependencies

3NF: No Transitive dependencies

BCNF: All determinants are candidate keys

Filled Prescription Relation

Filled Prescription (FilledPrescription_ID (Key), Prescription_ID (FK), Insurance_ID (FK), Pharmacy_ID (FK), Quantity_Dispensed, Date_Filled, Expiration, Instructions, Refills_Remaining)

Sample Data:

FilledPrescription_ID	Prescription_ID	Insurance_ID	Pharmacy_ID	Quantity_Dispensed	Date_Filled	Expiration	Instructions	Refills_Remaining
FP001	601000	INS001	PH001	30	01/16/2025	01/15/2026	Take with food	2
FP002	601001	INS002	PH002	60	02/11/2025	02/10/2026	Take twice daily	1
FP003	601002	INS003	PH003	60	03/06/2025	03/05/2026	Do not crush	0
FP004	601003	INS004	PH004	30	01/21/2025	01/20/2026	Take at bedtime	3
FP005	601004	INS005	PH005	60	02/26/2025	02/25/2026	Avoid alcohol	1

Key: FilledPrescription_ID

FD1: FilledPrescription_ID → Prescription_ID (FK), Insurance_ID (FK), Pharmacy_ID (FK),

Quantity_Dispensed, Date_Filled, Expiration, Instructions, Refills_Remaining

1NF: Meets the definition of a relation

2NF: No partial Key dependencies

3NF: No Transitive dependencies

BCNF: All determinants are candidate keys

Inventory Relation

Inventory (Inventory_ID (Key), Pharmacy_ID (FK), Medication_ID (FK), Date_Ordered,

Date_Received, Item_Name, Stock_keeping_Unit, Universal_Product_Code, Serial_Number,

Item_Location, Invoice_Price, Point_Of_Sale_Price, Quantity)

Sample Data:

Inventory_ID	Pharmacy_ID	Medication_ID	Date_Ordered	Date_Received	Item_Name	Stock_keeping_Unit	Universal_Product_Code	Serial_Number	Item_Location	Invoice_Price	Point_Of_Sale_Price	Quantity
INV001	PH001	MED001	01/05/2025	01/10/2025	Medication A	SKU1001	12345678901	SN001234	Shelf A1	50	75	500
INV002	PH002	MED002	01/10/2025	01/15/2025	Medication B	SKU2002	123456789012	SN002345	Shelf B2	30	45	300
INV003	PH003	MED003	01/15/2025	01/20/2025	Medication C	SKU3003	234567890123	SN003456	Shelf C3	20	35	400
INV004	PH004	MED004	01/20/2025	01/25/2025	Medication D	SKU4004	345678901234	SN004567	Shelf D4	75	100	250
INV005	PH005	MED005	01/25/2025	01/30/2025	Medication E	SKU5005	456789012345	SN005678	Shelf E5	15	25	600

Key: Inventory_ID

FD1: Inventory_ID → Pharmacy_ID (FK), Medication_ID (FK), Date_Ordered, Date_Received,

Item_Name, Stock_keeping_Unit, Universal_Product_Code, Serial_Number, Item_Location,

Invoice_Price, Point_Of_Sale_Price, Quantity

1NF: Meets the definition of a relation

2NF: No partial Key dependencies

3NF: No transitive dependency exists

Final Relations:

PatientData (Patient_ID (Key), FirstName, LastName, Date_of_Birth, Address, Suite,

Postal_Code (fk), Phone_Number, Email_Address, Social_Security_Number, Allergies)

Postal_Codes (Postal_Code (key), City, State_Province)

PrescriberData (Prescriber_ID (Key), FirstName, Prescriber_National_Provider_Identifier,

Street_Address, Suite, Postal_Code (fk), Phone_Number, Email_Address, Fax_Number)

Medication_Data(Medication_ID (Key), Lot_Number, Expiration_Date, Quantity, Serial_Number, National_Drug_Code (fk))

DrugCodes (National_Drug_Code (Key), Manufacturer, Strength, Size, Color, Imprint)

Prescription (Prescription_ID (Key), Patient_ID (FK), Prescriber_ID (FK), Medication_ID (FK), Date_Prescribed, Expiration, Refills, Start_Date, End_Date, Dosage, Quantity)

Insurance Plan (Insurance_ID (Key), Date_Of_Issue, Plan_Type, Member_ID, BIN, PCN, Issuer, RX_GroupNumber, Deductible, Copayment)

Pharmacy_Data(Pharmacy_ID (Key), Name, Phone_Number, Street_Address, Suite, Postal_Code (FK), License_Number, NPI_Number)

Filled Prescription (FilledPrescription_ID (Key), Prescription_ID (FK), Insurance_ID (FK), Pharmacy_ID (FK), Quantity_Dispensed, Date_Filled, Expiration, Instructions, Refills_Remaining)

InventoryData (Inventory_ID (Key), Pharmacy_ID (FK), Medication_ID (FK), Date_Ordered, Date_Received, Stock_Keeping_Unit, Universal_Product_Code (FK), Serial_Number, Item_Location, Invoice_Price, Point_Of_Sale_Price)

Commentary

- Removed Dosage from medication, didn't make sense
- Changed Drug_Code to National_Drug_Code
- Removed country where applicable, did not feel necessary and created complications
- Fixed spelling for conversion model

V. Creating the Database Schema with Structured Query Language

Create a table in the database for each of the relations in the final set of relations.

The following SQL code creates the ten tables and adds the PRIMARY KEY constraint to each one:

```
CREATE TABLE Patient
(
  Patient_ID VARCHAR(20) NOT NULL,
  FirstName VARCHAR(100) NOT NULL,
  LastName VARCHAR(100) NOT NULL,
  Date_of_Birth DATE ,
```

```
Address VARCHAR(50),
Suite VARCHAR(50),
Postal_Code VARCHAR(20) ,
Phone_Number VARCHAR(20),
Email_Address VARCHAR(100),
Social_Security_Number VARCHAR(11),
Allergies VARCHAR(255),
CONSTRAINT pk_patient
    PRIMARY KEY (Patient_ID)
);
```

```
CREATE TABLE Prescriber
(
    Prescriber_ID INTEGER NOT NULL,
    FirstName VARCHAR(100) NOT NULL,
    Prescriber_National_Provider_Identifier VARCHAR(20) NOT NULL,
    Street_Address VARCHAR(50),
    Suite VARCHAR(50),
    Postal_Code VARCHAR(20) ,
    Phone_Number VARCHAR(20),
    Email_Address VARCHAR(100),
    Fax_Number VARCHAR(20),
    CONSTRAINT pk_prescriber
        PRIMARY KEY (Prescriber_ID)
);
```

```
CREATE TABLE Medication
(
    Medication_ID INT NOT NULL,
    Lot_Number VARCHAR(100) NOT NULL,
    Expiration_Date DATE NOT NULL,
    Quantity INT NOT NULL,
    Serial_Number VARCHAR(100),
    National_Drug_Code VARCHAR(20) NOT NULL,
    CONSTRAINT pk_medication
        PRIMARY KEY (Medication_ID)
);
```

```
CREATE TABLE Insurance_Plan
(
    Insurance_ID INTEGER NOT NULL,
    Date_Of_Issue DATE NOT NULL,
    Plan_Type VARCHAR(50),
    Member_ID VARCHAR(50) NOT NULL,
```

```

    BIN VARCHAR(20),
    PCN VARCHAR(20),
    Issuer VARCHAR(100),
    RX_GroupNumber VARCHAR(50),
    Deductible VARCHAR(20),
    Copayment VARCHAR(20),
    CONSTRAINT pk_insurance_plan
        PRIMARY KEY (Insurance_ID)
);
CREATE TABLE Pharmacy
(
    Pharmacy_ID INT NOT NULL,
    Name VARCHAR(50) NOT NULL,
    Phone_Number VARCHAR(20),
    Street_Address VARCHAR(100),
    Suite VARCHAR(50),
    Postal_Code VARCHAR(20),
    License_Number VARCHAR(50) NOT NULL,
    NPI_Number VARCHAR(20),
    CONSTRAINT pk_pharmacy
        PRIMARY KEY (Pharmacy_ID)
);

CREATE TABLE Prescription
(
    Prescription_ID INT NOT NULL,
    Patient_ID INT NOT NULL,
    Prescriber_ID INT NOT NULL,
    Medication_ID INT NOT NULL,
    Date_Prescribed DATE,
    Expiration DATE,
    Refills INTEGER,
    Start_Date DATE,
    End_Date DATE,
    Dosage VARCHAR(100) NOT NULL,
    Quantity VARCHAR(20),
    CONSTRAINT pk_prescription
        PRIMARY KEY (Prescription_ID)
);

CREATE TABLE Filled_Prescription
(
    FilledPrescription_ID NUMBER NOT NULL,
    Prescription_ID NUMBER NOT NULL,

```

```
Insurance_ID VARCHAR(20) NOT NULL,  
Pharmacy_ID VARCHAR(20) NOT NULL,  
Quantity_Dispensed NUMBER,  
Date_Filled DATE NOT NULL,  
Expiration DATE,  
Instructions VARCHAR(200),  
Refills_Remaining NUMBER,  
CONSTRAINT pk_filled_prescription  
    PRIMARY KEY (FilledPrescription_ID)  
);
```

```
CREATE TABLE DrugCodes  
(  
    National_Drug_Code NUMBER NOT NULL,  
    Manufacturer VARCHAR(100),  
    Strength VARCHAR(50),  
    Size VARCHAR(50),  
    Color VARCHAR(50),  
    Imprint VARCHAR(100),  
    CONSTRAINT pk_drugcodes  
        PRIMARY KEY (National_Drug_Code)  
);
```

```
CREATE TABLE Inventory  
(  
    Inventory_ID NUMBER NOT NULL,  
    Pharmacy_ID NUMBER NOT NULL,  
    Medication_ID NUMBER NOT NULL,  
    Date_Ordered DATE,  
    Date_Received DATE,  
    Stock_Keeping_Unit VARCHAR(100),  
    Universal_Product_Code VARCHAR(50),  
    Serial_Number VARCHAR(100),  
    Item_Location VARCHAR(100),  
    Invoice_Price VARCHAR(20),  
    Point_Of_Sale_Price VARCHAR(30),  
    CONSTRAINT pk_inventory  
        PRIMARY KEY (Inventory_ID)  
);
```

Specifying Foreign Keys

The following SQL codes add FOREIGN KEY constraints to link the tables together:

```
ALTER TABLE Patient
  ADD CONSTRAINT fk_patient_postal_codes
  FOREIGN KEY (Postal_Code)
  REFERENCES Postal_Codes (Postal_Code);

ALTER TABLE Prescriber
  ADD CONSTRAINT fk_prescriber_postal_codes
  FOREIGN KEY (Postal_Code)
  REFERENCES Postal_Codes (Postal_Code);

ALTER TABLE Medication
  ADD CONSTRAINT fk_National_Drug_Code
  FOREIGN KEY (National_Drug_Code)
  REFERENCES DrugCodes (National_Drug_Code);

ALTER TABLE Pharmacy
  ADD CONSTRAINT fk_pharmacy_postal_codes
  FOREIGN KEY (Postal_Code)
  REFERENCES Postal_Codes (Postal_Code);

ALTER TABLE Prescription
  ADD CONSTRAINT fk_prescription_patient
  FOREIGN KEY (Patient_ID)
  REFERENCES Patient (Patient_ID);

ALTER TABLE Prescription
  ADD CONSTRAINT fk_prescription_prescriber
  FOREIGN KEY (Prescriber_ID)
  REFERENCES Prescriber (Prescriber_ID);

ALTER TABLE Prescription
  ADD CONSTRAINT fk_prescription_medication
  FOREIGN KEY (Medication_ID)
  REFERENCES Medication (Medication_ID);

ALTER TABLE Filled_Prescription
  ADD CONSTRAINT fk_filled_prescription_prescription
  FOREIGN KEY (Prescription_ID)
  REFERENCES Prescription (Prescription_ID);

ALTER TABLE Filled_Prescription
  ADD CONSTRAINT fk_filled_prescription_insurance_plan
```

```
FOREIGN KEY (Insurance_ID)
REFERENCES Insurance_Plan (Insurance_ID);
```

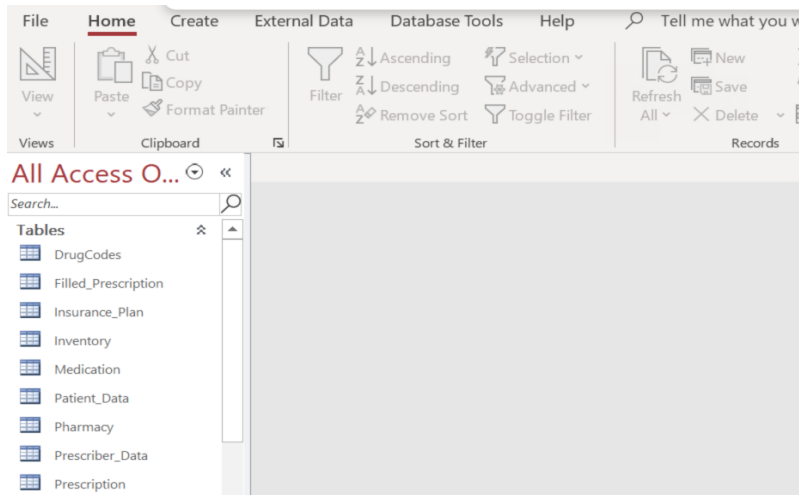
```
ALTER TABLE Inventory
ADD CONSTRAINT fk_inventory_pharmacy
FOREIGN KEY (Pharmacy_ID)
REFERENCES Pharmacy(Pharmacy_ID);
```

```
ALTER TABLE Inventory
ADD CONSTRAINT fk_inventory_medication
FOREIGN KEY (Medication_ID)
REFERENCES Medication(Medication_ID);
```

Commentary on SQL

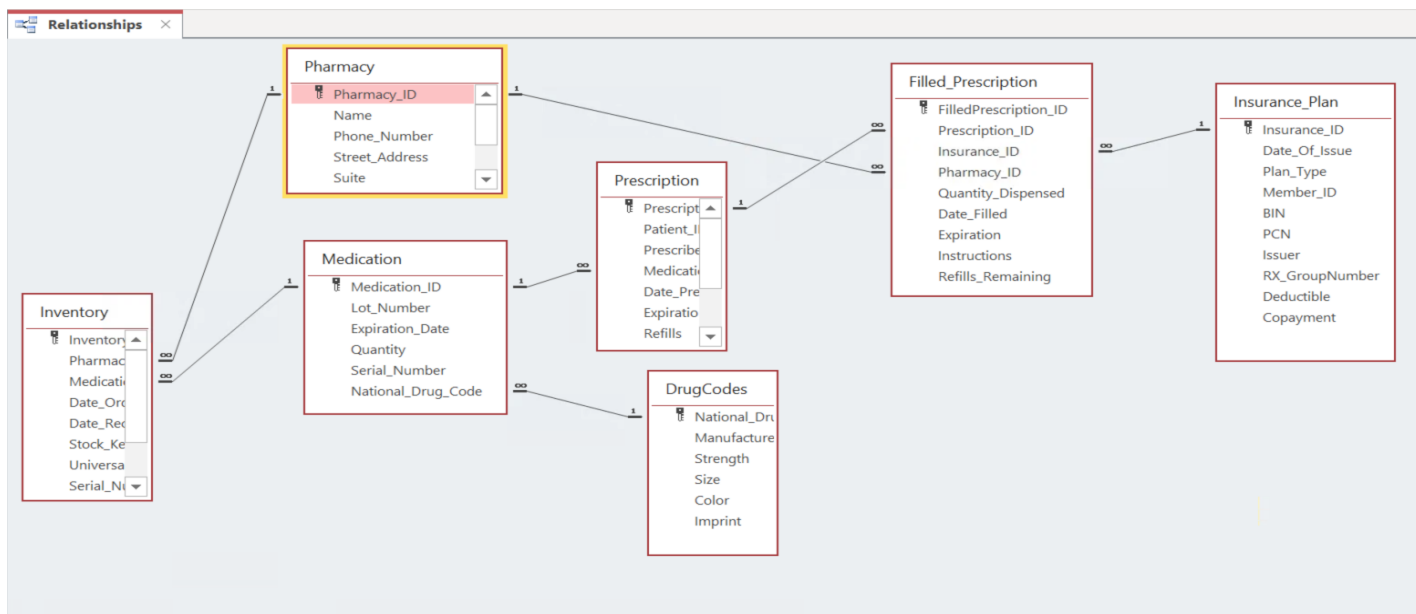
- Most DBMS will store the DATE and TIME in the same column. So Pickup_Date and Pickup_Time were combined into a single column named Pickup_DateTime in the FilledPrescription table.
- Foreign key columns are created in the appropriate tables. For example, when the FilledPrescription table is created, it includes the Prescription_ID, Pharmacy_ID, and Insurance_ID columns in the CREATE TABLE statement.
- Key and foreign key columns share the exact same name and data type across all tables. For example, Patient_ID is defined as VARCHAR(10) in the Patient table and as Patient_ID VARCHAR(10) in the Prescription table.
- Constraints are given meaningful names. For example, pk_patient is used for the primary key of the Patient table, and fk_filledprescription_patient is used for the foreign key linking FilledPrescription back to Patient.
- Columns such as Phone_Number, Postal_Code, BIN, and Social_Security_Number use the VARCHAR data type. If NUMBER or INTEGER were used instead, leading zeros would be lost — for example, a zip code of 01001 would be stored as 1001.

After creating the tables and adding the foreign key constraints, the database schema now looks like the following:



Relationship View

Using the Relationship View under Database Tools, we can see the relationships (foreign keys) between the tables:



Adding Data to the Tables using SQL INSERT Statements

```
INSERT INTO Drug_Codes (National_Drug_Code, Manufacturer, Strength, Size, Color, Imprint)
VALUES ('12345', 'Pfizer', '500mg', '5mm', 'Blue', 'L5');
INSERT INTO Drug_Codes (National_Drug_Code, Manufacturer, Strength, Size, Color, Imprint)
VALUES ('13345', 'Merck', '10mg', '10mm', 'White', 'M7');
INSERT INTO Drug_Codes (National_Drug_Code, Manufacturer, Strength, Size, Color, Imprint)
VALUES ('14345', 'Novartis', '250mg', '6mm', 'Yellow', 'N3');
INSERT INTO Drug_Codes (National_Drug_Code, Manufacturer, Strength, Size, Color, Imprint)
VALUES ('15345', 'Astrazenca', '50mg', '2mm', 'Pink', 'A9');
```

```
INSERT INTO Drug_Codes (National_Drug_Code, Manufacturer, Strength, Size, Color, Imprint)
VALUES ('16345', ' GSK ', '1000mg ', '8mm', 'Orange', 'G2');
```

```
INSERT INTO Filled_Prescription(FilledPrescription_ID, Prescription_ID, Insurance_ID,
Pharmacy_ID, Quantity_Dispensed, Date_Filled, Expiration, Instructions, Refills_Remaining)
VALUES ('5001', '1001 ', '4001 ', '2002', '90', '2/1/2001', '2/2/2003', ' Take with Food ', '2');
INSERT INTO Filled_Prescription(FilledPrescription_ID, Prescription_ID, Insurance_ID,
Pharmacy_ID, Quantity_Dispensed, Date_Filled, Expiration, Instructions, Refills_Remaining)
VALUES ('5002', '1002 ', '4002', '2003', '45', '8/10/2003', '2/5/2004', 'Take before meal ', '4');
INSERT INTO Filled_Prescription(FilledPrescription_ID, Prescription_ID, Insurance_ID,
Pharmacy_ID, Quantity_Dispensed, Date_Filled, Expiration, Instructions, Refills_Remaining)
VALUES ('5003', '1003 ', '4003', '2004', '30', '9/4/2004', '9/8/2006', ' Avoid Alcohol ', '1');
INSERT INTO Filled_Prescription(FilledPrescription_ID, Prescription_ID, Insurance_ID,
Pharmacy_ID, Quantity_Dispensed, Date_Filled, Expiration, Instructions, Refills_Remaining)
VALUES ('5004', '1004 ', '4004', '2005', '60', '10/9/2014', '9/4/2016', ' Take on empty Stomach ', '10');
INSERT INTO Filled_Prescription(FilledPrescription_ID, Prescription_ID, Insurance_ID,
Pharmacy_ID, Quantity_Dispensed, Date_Filled, Expiration, Instructions, Refills_Remaining)
VALUES ('5005', '1004 ', '4005', '2006', '30', '4/7/2011', '11/4/2014', ' As needed for pain ', '5');
```

```
INSERT INTO Insurance_Plan(Insurance_ID , Date_Of_Issue, Plan_Type, Member_ID, BIN, PCN,
Issuer, RX_GroupNumber, Deductible Copayment ) VALUES ( '4001', '2/2/2004', ' PPO ', '
MEM10001', ' 001100 ', ' PCN100', ' Blue Cross ', ' RXGRP01', '$500.00', '$10.00' );
INSERT INTO Insurance(Insurance_ID , Date_Of_Issue, Plan_Type, Member_ID, BIN, PCN, Issuer,
RX_GroupNumber, Deductible Copayment ) VALUES ( '4002', '3/6/2005', ' HMO ', ' MEM10002',
'2/2/2003', ' 002200 ', ' PCN101', ' Aetna ', ' RXGRP02', '$1,000.00', '$25.00' );
INSERT INTO Insurance(Insurance_ID , Date_Of_Issue, Plan_Type, Member_ID, BIN, PCN, Issuer,
RX_GroupNumber, Deductible Copayment ) VALUES ( '4003', '4/5/2004', ' EPO', ' MEM10003', '
003300 ', ' PCN102', ' Cigna ', ' RXGRP03', '$400.00', '$20.00' );
INSERT INTO Insurance(Insurance_ID , Date_Of_Issue, Plan_Type, Member_ID, BIN, PCN, Issuer,
RX_GroupNumber, Deductible Copayment ) VALUES ( '4004', '1/1/2010', ' POS', ' MEM10004', '
004400 ', ' PCN103', ' United Health ', ' RXGRP04', '$1,200.00', '$30.00' );
INSERT INTO Insurance(Insurance_ID , Date_Of_Issue, Plan_Type, Member_ID, BIN, PCN, Issuer,
RX_GroupNumber, Deductible Copayment ) VALUES ( '4005', '8/8/2018', ' High Deductible ', '
MEM10005', ' 005500 ', ' PCN104', ' Kaiser ', ' RXGRP05', '$2,500.00', '$50.00' );
```

```
INSERT INTO Inventory (Inventory_ID, Pharmacy_ID, Medication_ID, Date_Ordered,
Date_Received , Stock_Keeping_Unit, Universal_Product_Code, Serial_Number, Item_Location,
Invoice_Price, Point_Of_Sale_Price
) VALUES ( '3001', '2001', '1111', '12/1/2015', '12/3/2015', 'SKU123', 'UPC001', ' SN1001', 'Aisle4-
Shelf2', '$50.00', '$75.00', ); INSERT INTO Inventory (Inventory_ID, Pharmacy_ID, Medication_ID,
Date_Ordered, Date_Received , Stock_Keeping_Unit, Universal_Product_Code, Serial_Number,
Item_Location, Invoice_Price, Point_Of_Sale_Price
) VALUES ('3002', '2002', '2222', ' 6/4/2005', '8/10/2005 ', 'SKU456', ' UPC002', ' SN1002', 'Aisle2-
Shelf3', '$30.00', '$75.00');
```

```
INSERT INTO Inventory (Inventory_ID, Pharmacy_ID, Medication_ID, Date_Ordered,
Date_Received , Stock_Keeping_Unit, Universal_Product_Code, Serial_Number, Item_Location,
Invoice_Price, Point_Of_Sale_Price
```

```
) VALUES ('3003', '2003', '3333', '1/1/2000', '5/5/2000', 'SKU789', 'UPC003', 'SN1003', 'Aisle3-Shelf1', '$100.00', '$150.00');
```

```
INSERT INTO Inventory (Inventory_ID, Pharmacy_ID, Medication_ID, Date_Ordered, Date_Received, Stock_Keeping_Unit, Universal_Product_Code, Serial_Number, Item_Location, Invoice_Price, Point_Of_Sale_Price
```

```
) VALUES ('3004', '2004', '4444', '2/4/2010', '6/4/2014', 'SKU987', 'UPC004', 'SN1004', 'Aisle8-Shelf3', '$60.00', '$85.00');
```

```
INSERT INTO Inventory (Inventory_ID, Pharmacy_ID, Medication_ID, Date_Ordered, Date_Received, Stock_Keeping_Unit, Universal_Product_Code, Serial_Number, Item_Location, Invoice_Price, Point_Of_Sale_Price
```

```
) VALUES ('3005', '2005', '5555', '6/7/2012', '9/9/2015', 'SKU654', 'UPC005', 'SN1005', 'Aisle1-Shelf4', '$25.00', '$40.00');
```

```
INSERT INTO Medication (Medication_ID, Lot_Number, Expiration_Date, Quantity, Serial_Number, National_Drug_Code) VALUES ('1111', 'LOT123', '1/1/2006', '500', 'SN101', '12345');
```

```
INSERT INTO Medication (Medication_ID, Lot_Number, Expiration_Date, Quantity, Serial_Number, National_Drug_Code) VALUES ('2222', 'LOT456', '5/15/2016', '250', 'SN102', '13345');
```

```
INSERT INTO Medication (Medication_ID, Lot_Number, Expiration_Date, Quantity, Serial_Number, National_Drug_Code) VALUES ('3333', 'LOT789', '4/25/1999', '200', 'SN103', '14345');
```

```
INSERT INTO Medication (Medication_ID, Lot_Number, Expiration_Date, Quantity, Serial_Number, National_Drug_Code) VALUES ('4444', 'LOT001', '6/19/2015', '300', 'SN104', '15345');
```

```
INSERT INTO Medication (Medication_ID, Lot_Number, Expiration_Date, Quantity, Serial_Number, National_Drug_Code) VALUES ('5555', 'LOT002', '9/3/2005', '700', 'SN105', '16345');
```

```
INSERT INTO Patient_ID (Patient_ID, FirstName, LastName, Date_of_Birth, Address, Suite, Postal_Code, Phone_Number, Email_Address, Social_Security_Number, Allergies) VALUES ('01', 'John', 'Doe', '8/2/1925', '123 Main ST', '2D', '11001', '(718) 123-4567', 'john.doe@email.com', '123-45-6789', 'Penicillin');
```

```
INSERT INTO Patient_ID (Patient_ID, FirstName, LastName, Date_of_Birth, Address, Suite, Postal_Code, Phone_Number, Email_Address, Social_Security_Number, Allergies) VALUES ('02', 'Rose', 'Smith', '5/20/1945', '456 Oak Ave', '1st FL', '11011', '(917) 987-6543', 'rose.smith@email.com', '987-65-4032', 'Latex');
```

```
INSERT INTO Patient_ID (Patient_ID, FirstName, LastName, Date_of_Birth, Address, Suite, Postal_Code, Phone_Number, Email_Address, Social_Security_Number, Allergies) VALUES ('03', 'Dowsen', 'Robert', '3/10/1930', '789 Pine RD', '8A', '21001', '(888) 989-8777', 'dowsen.robert@email.com', '098-65-4321', 'Sulfa');
```

```
INSERT INTO Patient_ID (Patient_ID, FirstName, LastName, Date_of_Birth, Address, Suite, Postal_Code, Phone_Number, Email_Address, Social_Security_Number, Allergies) VALUES ('04', 'Maria', 'Garcia', '4/24/1976', '321 Elm ST', 'Unit 7', '21002', '(917) 345-6789', 'maria.gracia@email.com', '123-45-5678', 'Peanuts');
```

```
INSERT INTO Patient_ID (Patient_ID, FirstName, LastName, Date_of_Birth, Address, Suite, Postal_Code, Phone_Number, Email_Address, Social_Security_Number, Allergies) VALUES ('05', 'David', 'Lee', '1/5/1987', '654 Cedar Ln', 'Null', '21002', '(888) 098-7654', 'davis.lee@email.com', '876-09-8765', 'Null');
```

```
INSERT INTO Pharmacy (Pharmacy_ID, Name, Phone_Number, Street_Address, Suite, Postal_Code, License_Number, NPI_Number) VALUES ('2001', 'CVS', '(718) 101-2000', '10 Main ST', 'Suite 2', '11001', 'LIC201', '1122334455');
```

```
INSERT INTO Pharmacy(Pharmacy_ID, Name, Phone_Number , Street_Address, Suite, Postal_Code, License_Number, NPI_Number) VALUES ('2002', ' Duane Reade', '(888) 101-2100', '20 Oak Ave ', '1D', '11011', ' LIC202', '2233445566');
```

```
INSERT INTO Pharmacy(Pharmacy_ID, Name, Phone_Number , Street_Address, Suite, Postal_Code, License_Number, NPI_Number) VALUES ('2003', ' Rite Aid', '(855) 345-6789', '30 Pine Rd ', 'Ste B', '21001', ' LIC203', '3344556677');
```

```
INSERT INTO Pharmacy(Pharmacy_ID, Name, Phone_Number , Street_Address, Suite, Postal_Code, License_Number, NPI_Number) VALUES ('2004', ' Walmart ', '(888) 424-0400', '40 Cedan Dr ', '2B', '21002', ' LIC204', '4455667788');
```

```
INSERT INTO Pharmacy(Pharmacy_ID, Name, Phone_Number , Street_Address, Suite, Postal_Code, License_Number, NPI_Number) VALUES ('2005', 'Costo', '(917) 876-5432', '50 Rodeo Drive', '5R', '220021', ' LIC205', '5566778899');
```

```
INSERT INTO Prescriber_Data(Prescriber_ID, FirstName, Prescriber_National_Provider_Identifier, Street_Address, Suite, Postal_Code, Phone_Number, Email_Address, Fax_Number VALUES ('101', ' Sarah', '0102030405', '100 Medical Dr ', '4E', '11001', '(718) 098-0987', 'dr.sarah@medical.com ', '(718) 098-0978');
```

```
INSERT INTO Prescriber_Data(Prescriber_ID, FirstName, Prescriber_National_Provider_Identifier, Street_Address, Suite, Postal_Code, Phone_Number, Email_Address, Fax_Number VALUES ('102', ' Saulius', '9192939495', '345 Health Ln ', 'FL 12', '11011', '(845) 333-4545', 'saulius.md@health.com', '(800) 234-5689');
```

```
INSERT INTO Prescriber_Data(Prescriber_ID, FirstName, Prescriber_National_Provider_Identifier, Street_Address, Suite, Postal_Code, Phone_Number, Email_Address, Fax_Number VALUES ('103', ' Linda', '8283848586', '876 Care Ave ', 'suite 400', '21001', '(855) 987-3456', 'linda12@care.com', '(855) 987-3276');
```

```
INSERT INTO Prescriber_Data(Prescriber_ID, FirstName, Prescriber_National_Provider_Identifier, Street_Address, Suite, Postal_Code, Phone_Number, Email_Address, Fax_Number VALUES ('104', ' Aiden', '7172737475', '550 Recovery ST ', 'Null', '21002', '(877) 567-5432', 'aiden.dr@recovery.com ', '(718) 102-0400');
```

```
INSERT INTO Prescriber_Data(Prescriber_ID, FirstName, Prescriber_National_Provider_Identifier, Street_Address, Suite, Postal_Code, Phone_Number, Email_Address, Fax_Number VALUES ('105', ' Joseph', '6162636465', '678 Wellness RD', 'Ste 10', '21002', '(917) 876-5432', 'joseph.md@wellness.com', '(917) 876-5443');
```

```
INSERT INTO Prescription (Prescription_ID, Patient_ID, Prescriber_ID, Medication_ID, Date_Prescribed, Expiration, Refills, Start_Date, End_Date, Dosage, Quantity) VALUES ('1001', ' 01', '101', '1111', '2/1/2000', '4/1/2005', '2', '2/3/2000', '6/19/2000', 'Once daily', '90');
```

```
INSERT INTO Prescription (Prescription_ID, Patient_ID, Prescriber_ID, Medication_ID, Date_Prescribed, Expiration, Refills, Start_Date, End_Date, Dosage, Quantity) VALUES ('1002', ' 02', '102', '2222', '3/2/2000', '7/23/2000', '5', '3/5/2000', '3/23/2000', 'As Needed', '14');
```

```
INSERT INTO Prescription (Prescription_ID, Patient_ID, Prescriber_ID, Medication_ID, Date_Prescribed, Expiration, Refills, Start_Date, End_Date, Dosage, Quantity) VALUES ('1003', ' 03', '103', '3333', '8/20/2000', '6/27/2004', '0', '8/22/2026', '9/15/2026', 'Twice daily at Bedtime ', '45');
```

```
INSERT INTO Prescription (Prescription_ID, Patient_ID, Prescriber_ID, Medication_ID, Date_Prescribed, Expiration, Refills, Start_Date, End_Date, Dosage, Quantity) VALUES ('1004', ' 04', '104', '4444', '12/10/2000', '11/6/2002', '4', '12/14/2002', '4/9/2003', '1 tablet with juice', '30');
```

```
INSERT INTO Prescription (Prescription_ID, Patient_ID, Prescriber_ID, Medication_ID,
Date_Prescribed, Expiration, Refills, Start_Date, End_Date, Dosage, Quantity) VALUES ('1005', '05',
'105', '5555', '10/9/2000', '4/15/2001', '1', '6/1/2001', '8/2/2001', 'Half tablet at night ', '60');
```

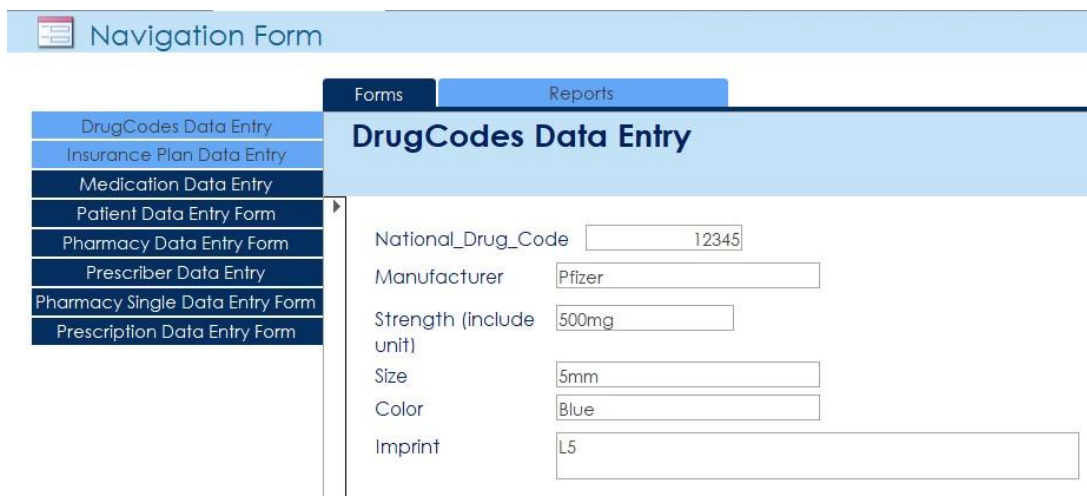
Now that the database schema has been created and is populated with some sample data, the database application can be developed.

VI. Database Application

The database application consists of a set of Forms, Reports, and Queries that are linked together on a Navigation Form.

The Navigation Form is the first form that appears when the database is opened.

Navigation Form



Different data entry forms and reports can be displayed by clicking on the selection on the left-hand side.

```
Private Sub Manufacturer_AfterUpdate ()
Manufacturer = StrConv (Manufacturer, vbProperCase)
End Sub
```

Patient Data Entry Form

Patient Data Entry Form

Patient_ID	01
FirstName	John
LastName	Doe
Date_of_Birth	8/2/1925
Address	123 Main ST
Suite	2D
City	New York
State	NY
Postal_Code	10001
Phone_Number	(718) 123-4567
Email_Address	john.doe@email.com
Social_Security_Number	123-45-6789
Allergies	Penicillin

The Patient Data Entry Form is used to create a new record for a patient. As with the Appointments form, a new Patient ID can be created by clicking in a blank Patient ID field after a new record is created.

```
Private Sub Allergies_AfterUpdate()
Allergies = StrConv(Allergies, vbProperCase)
End Sub
```

```
Private Sub City_AfterUpdate()
City = StrConv(City, vbProperCase)
End Sub
```

```
Private Sub FirstName_AfterUpdate()
FirstName = StrConv(FirstName, vbProperCase)
End Sub
```

```
Private Sub LastName_AfterUpdate()
LastName = StrConv(LastName, vbProperCase)
End Sub
```



```
Private Sub Form_AfterUpdate()
```

```
End Sub
```

```
Private Sub Name_AfterUpdate()
```

```
LastName = StrConv(LastName, vbProperCase)
```

```
End Sub
```

Insurance Plan Data Entry

Insurance_ID	4001
Date_Of_Issue	2/2/2004
Plan_Type	PPO
Member_ID	MEM10001
BIN	001100
PCN	PCN100
Issuer	Blue Cross
RX_GroupNumber	RXGRP01
Deductible	\$500.00
Copayment	\$10.00

The Insurance Data Entry Form is used to create a new record for a patient.

```
Private Sub Issuer_AfterUpdate()
```

```
Issuer = StrConv(Issuer, vbProperCase)
```

```
End Sub
```

Medication Data Entry

Medication_ID	2222
Medication_name	Ibuprofen
Lot_Number	LOT456
Expiration_Date	5/15/2016
Quantity	250
Serial_Number	SN102
National_Drug_Code	13345
Manufacturer	Merck
Strength (include u	10mg
Size	10mm
Color	White
Imprint	M7

Medication data entry form to record patients' medication.

```
Private Sub Color_AfterUpdate()  
Color = StrConv(Color, vbProperCase)  
End Sub
```

```
Private Sub Manufacturer_AfterUpdate()  
Manufacturer = StrConv(Manufacturer, vbProperCase)  
End Sub
```

```
Private Sub Medication_name_AfterUpdate()  
Medication_name = StrConv(Medication_name, vbProperCase)  
End Sub
```

Prescription Data Entry Form

Prescription_ID	1001	Dosage	Once daily
Patient_ID	01	Quantity	90
Prescriber_ID	101	Patient	John
Medication_ID	1111	First Name	Doe
Date_Prescribed	2/1/2000	Patient	Doe
Expiration	4/1/2005	Last Name	Doe
Refills	2	Serial_Number	SN101
Start_Date	2/3/2000	Prescriber	Sarah
End_Date	6/19/2000	First Name	Sarah

Filled_Prescription	Prescription_ID	Insurance_IC	Pharmacy_ID	Quantity_Dispensed	Date_Filled	Expiration
	1001	4001	2004	90	2/1/2001	2/2/2000
*	1001					

```
Private Sub LastName_AfterUpdate()  
    LastName = StrConv(LastName, vbProperCase)  
End Sub
```

```
Private Sub Patient_Data_FirstName_AfterUpdate()  
    Patient_Data_FirstName = StrConv(Patient_Data_FirstName, vbProperCase)  
End Sub
```

```
Private Sub Prescriber_Data_FirstName_AfterUpdate()  
    Prescriber_Data_FirstName = StrConv(Prescriber_Data_FirstName, vbProperCase)  
End Sub
```

Pharmacy Single Data Entry Form

Pharmacy_ID	2001
Name	CVS
Phone_Number	(718) 101-2000
Street_Address	10 Main ST
Suite	Suite 2
City	Boston
State	MA
Postal_Code	11001
License_Number	LIC201
NPI_Number	1122334455

Prescriber Data Entry

Prescriber_ID	101
FirstName	Sarah
LastName	Smith
Prescriber_National Provider_Identifier	0102030405
Street_Address	100 Medical Dr
Suite	4E
City	Boston
State	MA
Postal_Code	11001
Phone_Number	(718) 098-0987
Email_Address	dr.sarah@medical.com
Fax_Number	(718) 098-0978

```
Private Sub FirstName_AfterUpdate()  
    FirstName = StrConv(FirstName, vbProperCase)  
End Sub
```

```
Private Sub LastName_AfterUpdate()  
    LastName = StrConv(LastName, vbProperCase)  
End Sub
```

The purpose of this form is to enter different pharmacy and inventory details associated with a medication record. This form could also be used to manage and track medication stock for the pharmacy. The Pharmacy and the Medication can be selected from their respective fields and combo boxes as shown below:

This report provides a summary of the number of prescriptions associated with each patient. It displays each patient's subscriber ID, first name, last name, and the total count of prescriptions recorded in the system.

Based on query:

```
SELECT Prescriber_Data.Prescriber_ID, Prescriber_Data.FirstName,  
Prescriber_Data.LastName, Count(Prescription.Prescription_ID) AS Total_Prescriptions  
FROM Prescriber_Data INNER JOIN Prescription ON Prescriber_Data.Prescriber_ID =  
Prescription.Prescriber_ID  
GROUP BY Prescriber_Data.Prescriber_ID, Prescriber_Data.FirstName,  
Prescriber_Data.LastName  
ORDER BY Count(Prescription.Prescription_ID) DESC;
```

```
SELECT Medication.Medication_ID, Medication.Medication_Name,  
Count(Prescription.Prescription_ID) AS Times_Prescribed  
FROM Medication INNER JOIN Prescription ON Medication.Medication_ID =  
Prescription.Medication_ID  
GROUP BY Medication.Medication_ID, Medication.Medication_Name  
ORDER BY Count(Prescription.Prescription_ID) DESC;
```

```
SELECT Pharmacy.Pharmacy_ID, Pharmacy.Name,  
Count(Filled_Prescription.Prescription_ID) AS Total_Filled  
FROM Pharmacy INNER JOIN Filled_Prescription ON Pharmacy.Pharmacy_ID =  
Filled_Prescription.Pharmacy_ID
```

```
GROUP BY Pharmacy.Pharmacy_ID, Pharmacy.Name
ORDER BY Count(Filled_Prescription.Prescription_ID) DESC;
```

Inventory ID	Inventory Item	Inventory Item Description	Stock Keeping Unit	Universal Inventory	Serial Number	Item Location	Original Price	Current Price	Inventory ID	Name	Phone	Street Address	Suite	City	State	Postal Code	License	NPI	Medical License	Lot Number	
3001	2001	1111	###	###	SKU123	UPC001	SN1001	Aisle4-Shelf2	\$50.00	\$75.00	2001	CVS	(718) 10 Main St	Suite 2	Boston	MA	11001	LIC201	11223	1111	LOT123
3002	2002	2222	###	###	SKU456	UPC002	SN1002	Aisle2-Shelf3	\$30.00	\$75.00	2002	Duane F	(888) 20 Oak Ave	1D	Miami	FL	11011	LIC202	22334	2222	LOT456
3003	2003	3333	###	###	SKU789	UPC003	SN1003	Aisle3-Shelf1	\$100.00	\$150.00	2003	Rite Aid	(855) 30 Pine Rd	Ste B	Austin	TX	21001	LIC203	33445	3333	LOT789
3004	2004	4444	###	###	SKU987	UPC004	SN1004	Aisle8-Shelf3	\$60.00	\$85.00	2004	Walmart	(888) 40 Cedar Dr	2B	Denver	CO	21002	LIC204	44556	4444	LOT001
3005	2005	5555	###	###	SKU654	UPC005	SN1005	Aisle1-Shelf4	\$25.00	\$40.00	2005	Costco	(917) 50 Rodeo Drive	5R	Portland	OR	22002	LIC205	55667	5555	LOT002

Friday, May 8, 2026

Patient	FirstName	LastName	Birth	Address	Suite	City	State	Postal	Phone_Numb	Email_Address	Social_Security	Allergies	U_ID	Prescrip	Prescrip	Prescrip	bed	ion	fills	ate	ate	Dosage	Quantity	Medication	Lot	
01	John	Doe	##	123 Mc	2D	New	NY	1000	(718) 123-4567	john.doe@er	123-45-6789	Penicillin		##	01	101	1111	###	##	2	##	##	Once daily	90	1111	LO
02	Rose	Smith	##	456 Oc	1st Fl	Garfi	NJ	0702	(917) 987-6543	rose.smith@er	987-65-4032	Latex		##	02	102	2222	###	##	5	##	##	As Needed	14	2222	LO
03	Dowsen	Robert	##	789 Pin	8A	Beve	CA	90211	(888) 989-8777	dowsen.robe	098-65-4321	Sulfa		##	03	103	3333	###	##	0	##	##	Twice daily at	45	3333	LO
04	Maria	Garcia	##	321 Elm	Unit 7	Chici	IL	6060	(917) 345-6789	maria.gracia	123-45-5678	Peanuts		##	04	104	4444	###	##	4	##	##	1 tablet with j	30	4444	LO
05	David	Lee	##	654 Ce		Hous	TX	7700	(888) 098-7654	davis.lee@er	876-09-8765	Null		##	05	105	5555	###	##	1	##	##	Half tablet at	60	5555	LO

Friday, May 8, 2026

Insurance_Plan

Insurance Issue	Plan Type	Member BIN	PCN	Issuer	RX_Group	deductible	copayment	option_ID	Prescrip	Filled_Pre	Filled_Pre	sponsored	filled	Instructions	maining acy_ID	Name	Phone	Street	City	State	Zip	
4001	### PPO	MEM10X 001100	PCN1	Blue Cross	RXGRP0	\$500.00	\$10.00	5001	1001	4001	2004	90	###	### Take with Food	2	2004	Walmar	(888)	40	Cedar	TX	75001
4002	### HMO	MEM10X 002200	PCN1	Aetna	RXGRP0	#####	\$25.00	5002	1002	4002	2004	45	###	### Take before me	4	2004	Walmar	(888)	40	Cedar	TX	75001
4003	### EPO	MEM10X 003300	PCN1	Cigna	RXGRP0	\$400.00	\$20.00	5003	1003	4003	2003	30	###	### Avoid Alcohol	1	2003	Rite Aid	(855)	30	Pine	TX	75001
4004	### POS	MEM10X 004400	PCN1	United Health	RXGRP0	#####	\$30.00	5004	1004	4004	2004	60	###	### Take on empty S	10	2004	Walmar	(888)	40	Cedar	TX	75001
4005	### High De	MEM10X 005500	PCN1	Kaiser	RXGRP0	#####	\$50.00	5005	1002	4005	2005	30	###	### As needed for p	5	2005	Costo	(917)	50	Rode	TX	75001

Friday, May 8, 2026

VII. Conclusions

Developing a database application follows a system development lifecycle that begins with conceptual modeling and moves through a structured set of steps that include logical modeling, normalization, physical implementation, and application development. The Pharmacy Management System project illustrates each of these major steps. The system was designed to manage and track critical healthcare data including patient records, prescriptions, medications, pharmacy inventory, and provider information. However, some details have not been fully documented. For example, some additional work may be required to further normalize the Prescription and Inventory relations to account for varying medication dosages and stock levels across multiple pharmacy locations. Additional application implementation details such as other VBA codes, more detailed descriptions of the use of each form and report, and enhanced security measures to protect sensitive patient data such as Social Security Numbers and medical histories could also be added.